# UNIT-3

# SYNTAX DIRECTED DEFINITIONS

# LECTURE- 1

Syntax directed definition specifies the values of attributes by associating semantic rules with the grammar productions.

It is a context free grammar with attributes and rules together which are associated with grammar symbols and productions respectively.

The process of syntax directed translation is two-fold:

- Construction of syntax tree and
- Computing values of attributes at each node by visiting the nodes of syntax tree.

## Semantic actions

Semantic actions are fragments of code which are embedded within production bodies by syntax directed translation.

They are usually enclosed within curly braces ({ }).

It can occur anywhere in a production but usually at the end of production.

(eg.)

$$E \rightarrow E_1 + T \{print\ '+'\}$$

## Types of translation:

**• L-attributed translation**

o It performs translation during parsing itself.

o No need of explicit tree construction.

o L represents 'left to right'.

**• S-attributed translation**

o It is performed in connection with bottom up parsing.

o 'S' represents synthesized.

## Types of attributes

**• Inherited attributes**

o It is defined by the semantic rule associated with the production at the parent of ‘node.

o Attributes values are confined to the parent of node, its siblings and by itself.

o The non-terminal concerned must be in the body of the production.

• **Synthesized attributes**

o It is defined by the semantic rule associated with the production at the node.

o Attributes values are confined to the children of node and by itself.

o The non terminal concerned must be in the head of production.

o Terminals have synthesized attributes which are the lexical values (denoted by lexval) generated by the lexical analyzer.

## Syntax directed definition of simple desk calculator

| Production | Semantic rules |
|---|---|
| L —> $E_n$ | *L.val = E.val* |
| E —> $E_1$+ T | *E.val = $E_1$.val+ T.val* |
| E —> T | *E.val = T.val* |
| T—> $T_1$*F | *T.val = $T_i$.val x F.val* |
| T —> F | *T.val = F.val* |
| F —> (E) | *F.val = E.val* |
| F —> digit | *F.val = digit.lexval* |

## Syntax-directed definition-inherited attributes

| Production | Semantic Rules |
|---|---|
| D —>TL | *L.inh = T.type* |
| T —> *int* | *T.type* =integer |
| T —> *float* | *T.type = float* |

| | |
|---|---|
| L —> $L_1$, **id** | $L_1$.inh = *L.inh*<br>*addType (id.entry, Linh)* |
| L —> **id** | *addType (id.entry, L.inh)* |

• Symbol T is associated with a synthesized attribute *type.*

• Symbol L is associated with an inherited attribute *inh,*

# LECTURE 2

# TYPES OF SYNTAX DIRECTED DEFINITIONS

## S-attributed Definitions

Syntax directed definition that involves only synthesized attributes is called S-attributed. Attribute values for the non-terminal at the head is computed from the attribute values of the symbols at the body of the production.

The attributes of a S-attributed SDD can be evaluated in bottom up order of nodes of the parse tree. i.e., by performing post order traversal of the parse tree and evaluating the attributes at a node when the traversal leaves that node for the last time.

| Production | Semantic rules |
|---|---|
| L —> $E_n$ | *L.val = E.val* |
| E —> $E_1$+ T | *E.val = $E_1$.val+ T.val* |
| E —> T | *E.val = T.val* |
| T—> $T_1$*F | *T.val = $T_i$.val x F.val* |
| T —> F | *T.val = F.val* |
| F —> (E) | *F.val = E.val* |
| F —> digit | *F.val = digit.lexval* |

## L-attributed Definitions

The syntax directed definition in which the edges of dependency graph for the attributes in production body, can go from left to right and not from right to left is called L-attributed definitions. Attributes of L-attributed definitions may either be synthesized or inherited.

If the attributes are inherited, it must be computed from:

• Inherited attribute associated with the production head.

• Either by inherited or synthesized attribute associated with the production located to the left of the attribute which is being computed.

Either by inherited or synthesized attribute associated with the attribute under consideration in such a way that no cycles can be formed by it in dependency graph.

| Production | Semantic Rules |
|---|---|
| T —> FT' | *T '.inh = F.val* |
| T ' —> *FT$_1$' | T'$_1$*.inh =T'.inh x F.val* |

In production **1,** the inherited attribute T' is computed from the value of F which is to its left.

In production 2, the inherited attributed Tl' is computed from T'. *inh* associated with its head and the value of F which appears to its left in the production. i.e., for computing inherited attribute it must either use *from the above* or *from the left* information of SDD.

# LECTURE-3

# TOP-DOWN TRANSLATION

## Example:

Remove left recursion

E-> E+T

E-> E-T

E -> T

T-> (E)

T-> int



E -> TR

R -> +TR

R -> -TR

R -> €

T-> (E)

T-> int

## SDT for the LL(1) grammar:

E -> E + T

{ $0.val = $1.val + $3.val; }

E -> E – T

{ $0.val = $1.val - $3.val; }

E -> T

{ $0.val = $1.val; }

E -> T R

{$2.in = $1.val; $0.val =$2.val; }



R -> + T R

{$3.in = $0.in + $2.val;

$0.val = $3.val; }

R -> - T R

{$3.in = $0.in - $2.val;

$0.val = $3.val; }

T -> **(** E **)**

{ $0.val = $0.in; }

T -> **int**

{ $0.val = $1.lexval; }

R -> €

{ $0.val = $0.in; }

T -> **(** E **)**

{ $0.val = $2.val; }

T -> **int**

{ $0.val = $1.lexval; }

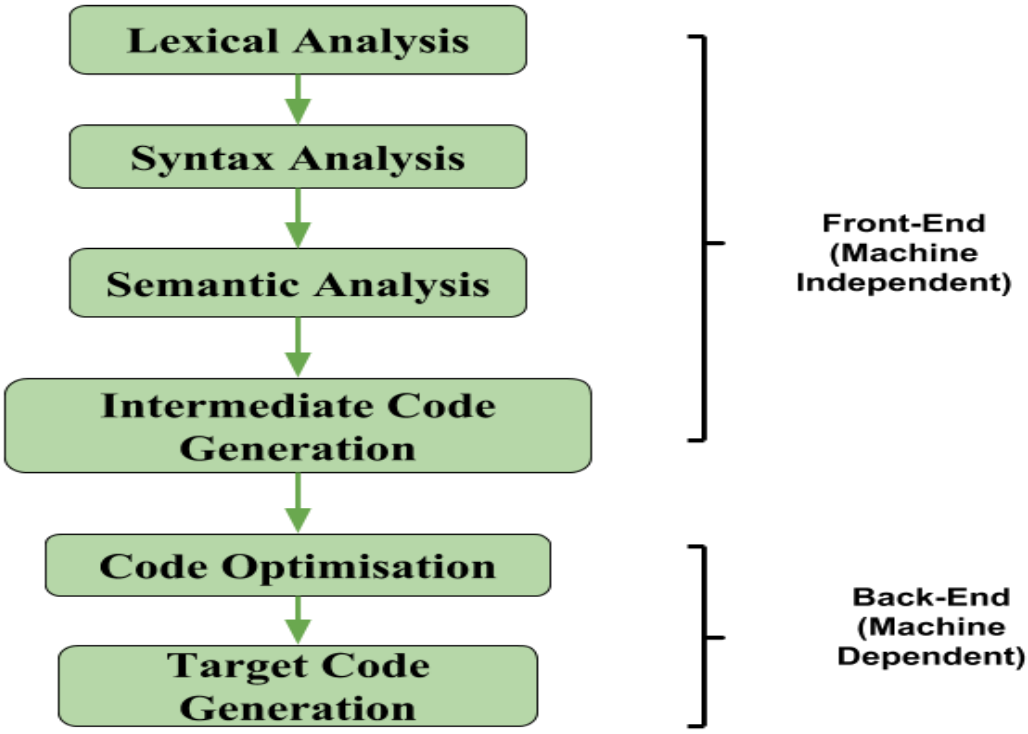
# LECTURE -4

# INTERMEDIATE CODE

## Intermediate code forms using postfix notation

In the analysis-synthesis model of a compiler, the front end of a compiler translates a source program into an independent intermediate code, then the back end of the compiler uses this intermediate code to generate the target code (which can be understood by the machine). The benefits of using machine-independent intermediate code are:

- Because of the machine-independent intermediate code, portability will be enhanced. For ex, suppose, if a compiler translates the source language to its target machine language without having the option for generating intermediate code, then for each new machine, a full native compiler is required. Because, obviously, there were some modifications in the compiler itself according to the machine specifications.
- Retargeting is facilitated.
- It is easier to apply source code modification to improve the performance of source code by optimizing the intermediate code.




If we generate machine code directly from source code then for n target machine we will have optimizers and n code generator but if we will have a machine-independent intermediate code, we will have only one optimizer. Intermediate code can be either language-specific (e.g., Bytecode for Java) or language. independent (three-address code).

## The following are commonly used intermediate code representations:

1. **Postfix Notation:** Also known as reverse Polish notation or suffix notation. The ordinary (infix) way of writing the sum of a and b is with an operator in the middle: a + b The postfix notation for the same expression places the operator at the right end as ab +. In general, if e1 and e2 are any postfix expressions, and + is any binary operator, the result of applying + to the values denoted by e1 and e2 is postfix notation by e1e2 +. No parentheses are needed in postfix notation because the position and arity (number of arguments) of the operators permit only one way to decode a postfix expression. In postfix notation, the operator follows the operand.

   **Example 1:** The postfix representation of the expression (a + b) * c is : ab + c *

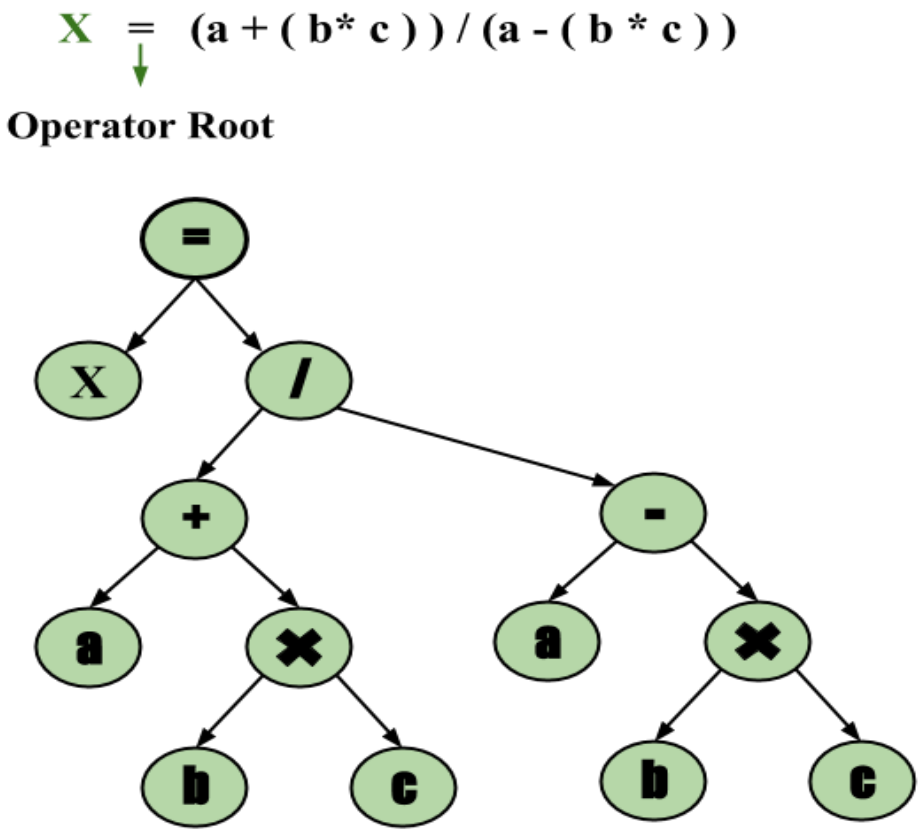
   **Example 2:** The postfix representation of the expression (a – b) * (c + d) + (a – b) is : ab – cd + *ab -+

2. **Three-Address Code:** A statement involving no more than three references(two for operands and one for result) is known as a three address statement. A sequence of three address statements is known as a three address code. Three address statement is of form x = y op z, where x, y, and z will have address (memory location). Sometimes a statement might contain less than three references but it is still called a three address statement.
   **Example:** The three address code for the expression a + b * c + d :
   T 1 = b * c
   T 2 = a + T 1
   T 3 = T 2 + d
   T 1 , T 2 , T 3 are temporary variables.
   There are 3 ways to represent a Three-Address Code in compiler design:

   i)Quadruples
   ii)Triples
   iii) Indirect Triples

3. **Syntax Tree:** A syntax tree is nothing more than a condensed form of a parse tree. The operator and keyword nodes of the parse tree are moved to their parents and a chain of single productions is replaced by the single link in the syntax tree the internal nodes are operators and child nodes are operands. To form a syntax tree put parentheses in the expression, this way it's easy to recognize which operand should come first.

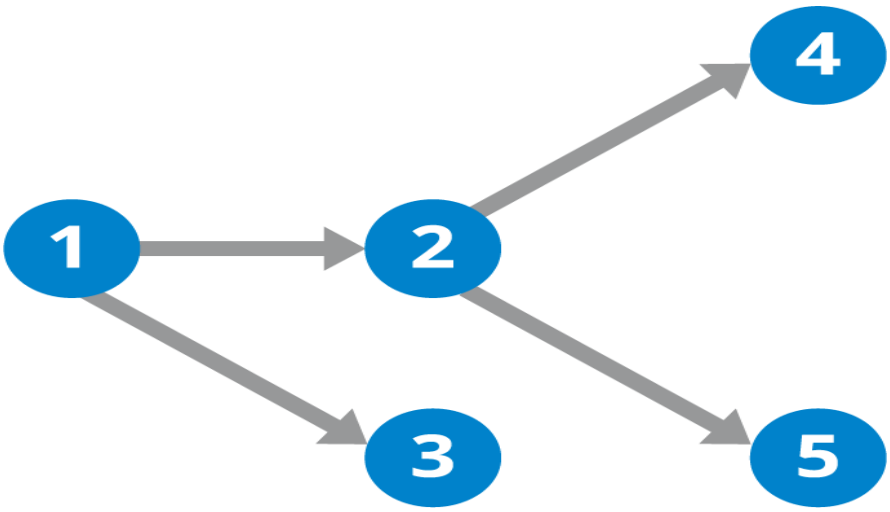
**Example:** x = (a + b * c) / (a – b * c)



# LECTURE -5

# DIRECTED ACYCLIC GRAPH

**Directed Acyclic Graph (DAG)** is a conceptual representation of a series of activities. The order of the activities is depicted by a graph, which is visually presented as a set of circles, each one representing an activity, some of which are connected by lines, which represent the flow from one activity to another. Each circle is known as a "vertex" and each line is known as an "edge." "Directed" means that each edge has a defined direction, so each edge necessarily represents a single directional flow from one vertex to another. "Acyclic" means that there are no loops (i.e., "cycles") in the graph, so that for any given vertex, if you follow an edge that connects that vertex to another, there is no path in the graph to get back to that initial vertex.




## Why Are Directed Acyclic Graphs Useful?

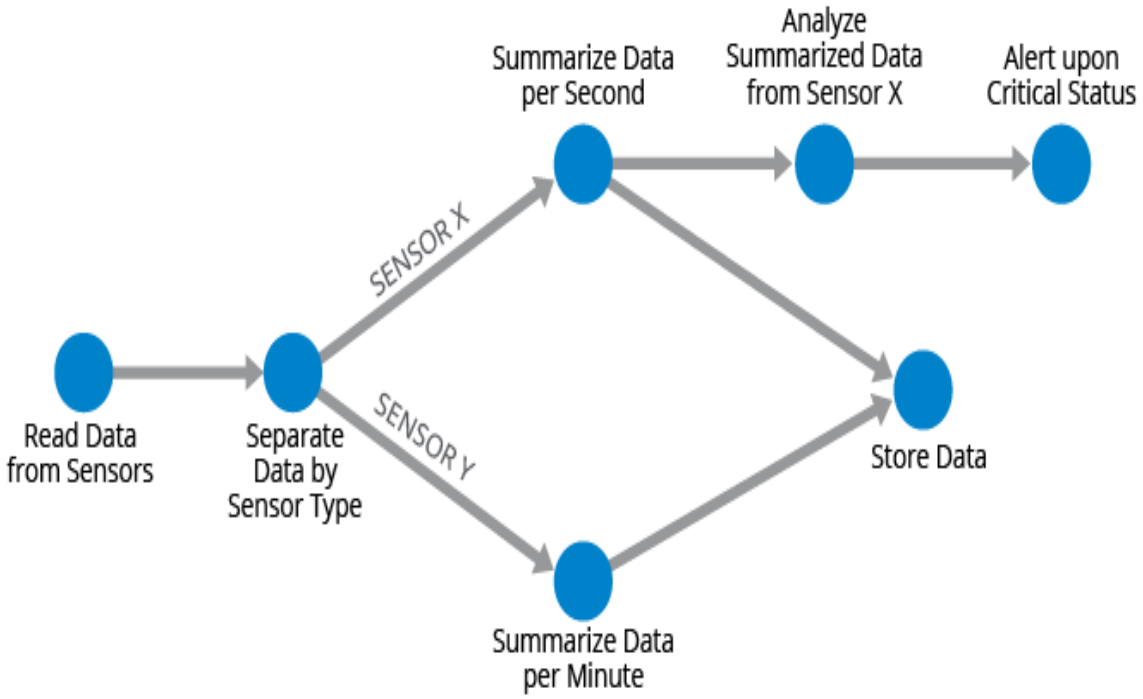
DAGs are useful for representing many different types of flows, including data processing flows. By thinking about large-scale processing flows in terms of DAGs, one can more clearly organize the various steps and the associated order for these jobs.

In many data processing environments, a series of computations are run on the data to prepare it for one or more ultimate destinations. This type of data processing flow is often referred to as a **data pipeline**. As an example, sales transaction data might be processed immediately to prepare it for making real-time recommendations to consumers.

As part of the processing lifecycle, the data can go through many steps including cleansing (correcting incorrect/invalid data), aggregation (calculating summaries), enrichment (identifying relationships with other relevant data), and transformation (writing the data into a new format).

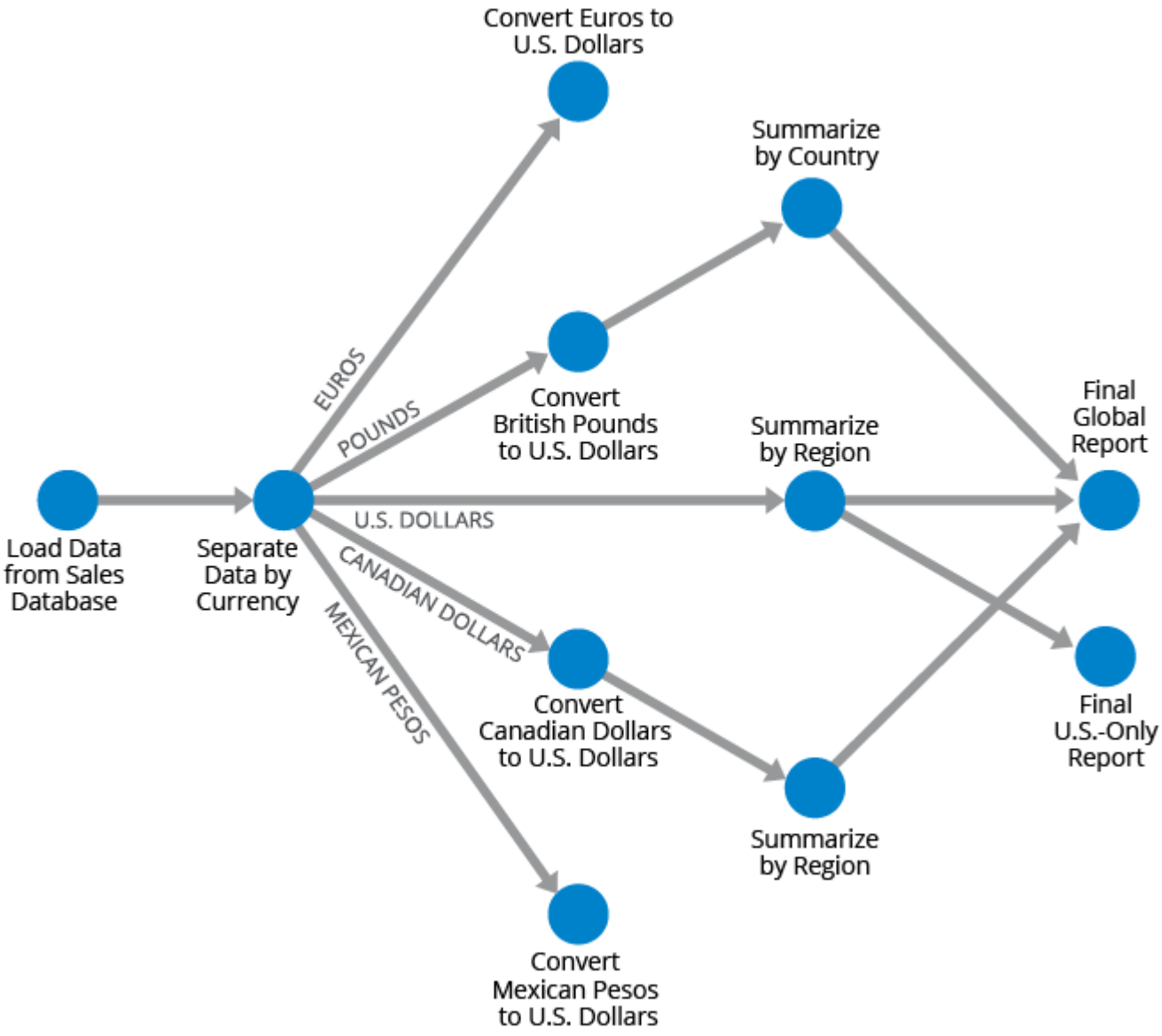
One **key characteristic** of DAGs and the data processing flows that they model is that there can be multiple paths in the flow. This is important because it recognizes the need to process data in multiple ways to accommodate different outputs and needs. In the example flow below, a stream of sensor data is processed.

The data is first loaded from the sensors, then are separated by the sensor type. Sensor X data will be summarized per second, and then analyzed in real-time. If any critical status is observed, an alert is sent. The data is also saved for long-term storage and possibly other analysis. Also in this flow is data from sensor Y, which for now is summarized per minute, and then stored in the same long-term store as the data for sensor X.




A stream of sensor data represented as a directed acyclic graph.

To give an example of how DAGs apply to **batch processing** pipelines, suppose you have a database of global sales, and you want a report of all sales by region, expressed in U.S. dollars. You might first load all data into a processing engine, separate out data by the different currencies, convert the financial figures to U.S. dollars, summarize the data by country/region, then bring all the data together into a final report. And let's say that the U.S.-only data will be created into a separate report as well. This data flow could be represented by the DAG shown below.




## Global sales data represented by the directed acyclic graph.

Since DAGs apply to both stream and batch processing, it is increasingly common to have hybrid data processing environments that handle both stream and batch data sets. Technologies such as **Hazelcast Jet** that are designed to handle both types of data let companies build data architectures that take advantage of all of their data.

# LECTURE- 6

# THREE ADDRESS CODE IN COMPILER

**Three address code** is a type of intermediate code which is easy to generate and can be easily converted to machine code.It makes use of at most three addresses and one operator to represent an expression and the value computed at each instruction is stored in temporary variable generated by compiler. The compiler decides the order of operation given by three address code.

## General representation : `a = b op c`

Where a, b or c represents operands like names, constants or compiler generated temporaries and op represents the operator.

**Example-1:** Convert the expression a * – (b + c) into three address code.

$$t_1 = b + c$$
$$t_2 = \text{uminus } t_1$$
$$t_3 = a * t_2$$

## Common Three Address Instruction Forms-

The common forms of Three Address instructions are-

### 1. Assignment Statement-

| x = y op z and x = op y |
|---|

Here,

- x, y and z are the operands.
- op represents the operator.

It assigns the result obtained after solving the right side expression of the assignment operator to the left side operand.

**2. Copy Statement-**

| x = y |
|---|

Here,

- x and y are the operands.
- = is an assignment operator.

It copies and assigns the value of operand y to operand x.

**3. Conditional Jump-**

| If x relop y goto X |
|---|

Here,

- x & y are the operands.
- X is the tag or label of the target statement.
- relop is a relational operator.

If the condition "x relop y" gets satisfied, then-

- The control is sent directly to the location specified by label X.
- All the statements in between are skipped.

If the condition "x relop y" fails, then-

- The control is not sent to the location specified by label X.
- The next statement appearing in the usual sequence is executed.

### 4. Unconditional Jump-

| goto X |
|---|

Here, X is the tag or label of the target statement.

On executing the statement,

- The control is sent directly to the location specified by label X.
- All the statements in between are skipped.

### 5. Procedure Call-

| param x call p return y |
|---|

Here, p is a function which takes x as a parameter and returns y.

# LECTURE- 7

# TAC FOR VARIOUS CONTROL STRUCTURES

**Backpatching** is basically a process of fulfilling unspecified information. This information is of labels. It basically uses the appropriate semantic actions during the process of code generation. It may indicate the address of the Label in goto statements while producing TACs for the given expressions. Here basically two passes are used because assigning the positions of these label statements in one pass is quite challenging. It can leave these addresses unidentified in the first pass and then populate them in the second round. Backpatching is the process of filling up gaps in incomplete transformations and information.

## Need for Backpatching:

Backpatching is mainly used for two purposes:

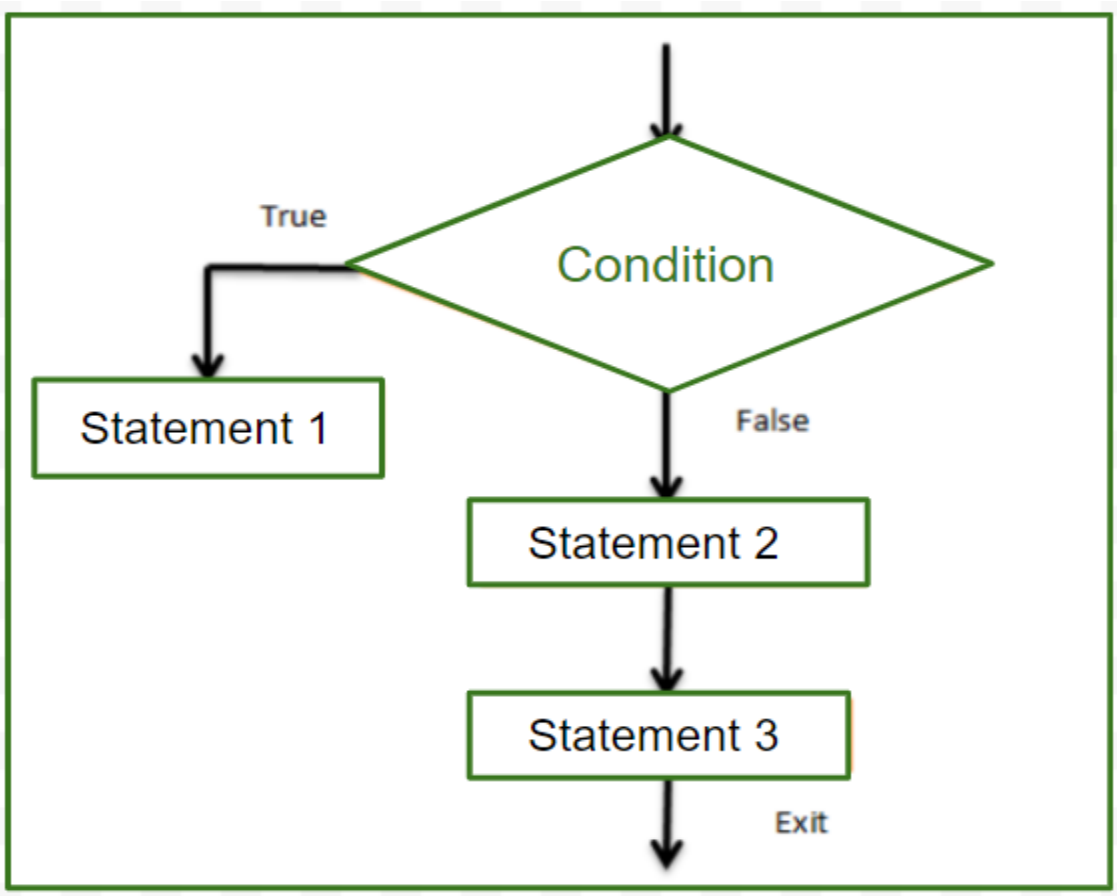
### 1. Boolean expression:

Boolean expressions are statements whose results can be either true or false. A boolean expression which is named for mathematician George Boole is an expression that evaluates to either true or false. Let's look at some common language examples:

- My favorite color is blue. → true
- I am afraid of mathematics. → false
- 2 is greater than 5. → false

### 2. Flow of control statements:

The flow of control statements needs to be controlled during the execution of statements in a program. For example,




*The flow of control statements*

## 3. Labels and Gotos:

The most elementary programming language construct for changing the flow of control in a program is a label and ***goto***. When a compiler encounters a statement like ***goto L***, it must check that there is exactly one statement with label ***L*** in the scope of this ***goto*** statement. If the label has already appeared, then the symbol table will have an entry giving the compiler-generated label for the first three-address instruction associated with the source statement labeled ***L***. For the translation, we generate a ***goto*** three-address statement with that compiler-generated label as a target.

When a label **L** is encountered for the first time in the source program, either in a declaration or as the target of the forward goto, we enter **L** into the symbol table and generate a symbolic table for **L**.

# One-pass code generation using backpatching:

In a single pass, backpatching may be used to create a boolean expressions program as well as the flow of control statements. The synthesized properties truelist and falselist of non-terminal B are used to handle labels in jumping code for Boolean statements. The label to which control should go if B is true should be added to B.truelist, which is a list of a jump or conditional jump instructions. B.falselist is the list of instructions that eventually get the label to which control is assigned when B is false. The jumps to true and false exist, as well as the label field, are left blank when the program is generated for B. The lists B.truelist and B.falselist, respectively, contain these early jumps.

A statement S, for example, has a synthesized attribute S.nextlist, which indicates a list of jumps to the instruction immediately after the code for S. It can generate instructions into an instruction array, with labels serving as indexes. We utilize three functions to modify the list of jumps:

- **Makelist (i):** Create a new list including only i, an index into the array of instructions and the makelist also returns a pointer to the newly generated list.
- **Merge(p1,p2):** Concatenates the lists pointed to by p1, and p2 and returns a pointer to the concatenated list.
- **Backpatch (p, i):** Inserts i as the target label for each of the instructions on the record pointed to by p.

# LECTURE- 8

## BACKPATCHING FOR BOOLEAN EXPRESSIONS

Using a translation technique, it can create code for Boolean expressions during bottom-up parsing. In grammar, a non-terminal marker M creates a semantic action that picks up the index of the next instruction to be created at the proper time.

**For Example,** Backpatching using boolean expressions production rules table:

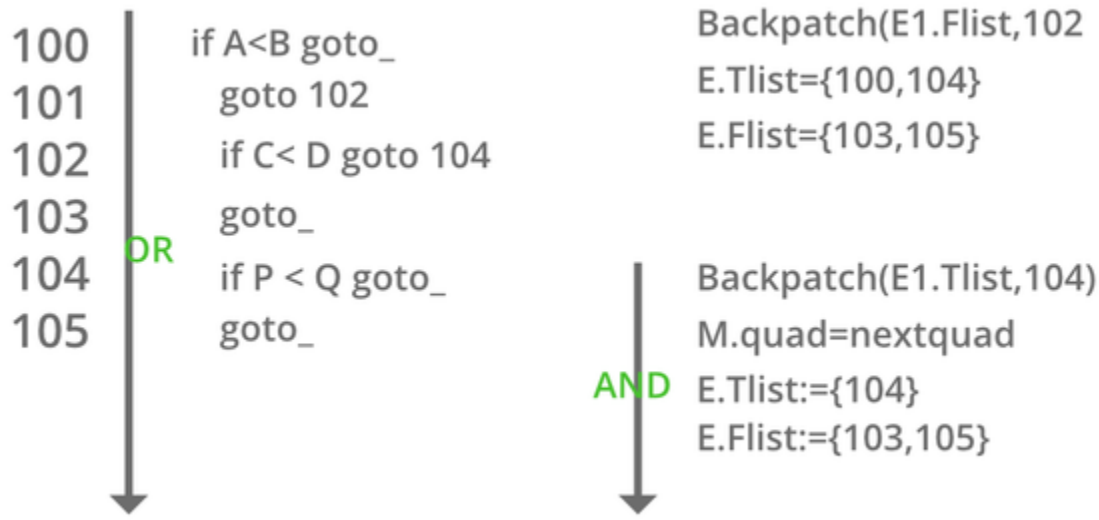
**Step 1:** Generation of the production table

| Production Rule | | Semantic action |
|---|---|---|
| E | $E_1$ OR M $E_2$ | {backpatch ( $E_1$.flist, M.quad); E.Tlist:=merge( $E_1$.Tlist, $E_2$.Tlist) E.Flist:= $E_2$.Flist} |
| E | $E_1$ AND M $E_2$ | {backpatch ( $E_1$.Tlist, M.quad); E.Tlist:=$E_2$.Tlist; E.Flist:=merge($E_1$.Flist,$E_2$.Flist);} |
| E | NOT $E_1$ | {E.Tlist:=$E_1$.Flist; E.Flist:=$E_1$.Tlist;} |
| E | ($E_1$) | {E.Tlist:=$E_1$.Tlist; E.Flist:=$E_1$.Flist;} |
| E | id1 relop id2 | {E.Tlist:=mklist(nextstate); E.Flist:=mklist(nextstate+); Append('if' id1.place relop.op id2.place 'goto_'); Append('goto_')} |
| E | true | {E.Tlist:=mklist(nextstate); Append('goto_');} |
| E | false | {E.Flist:=mklist(nextstate); Append('goto_');} |
| M | ε | {m.quad:=nextquad;} |

*Production Table for Backpatching*

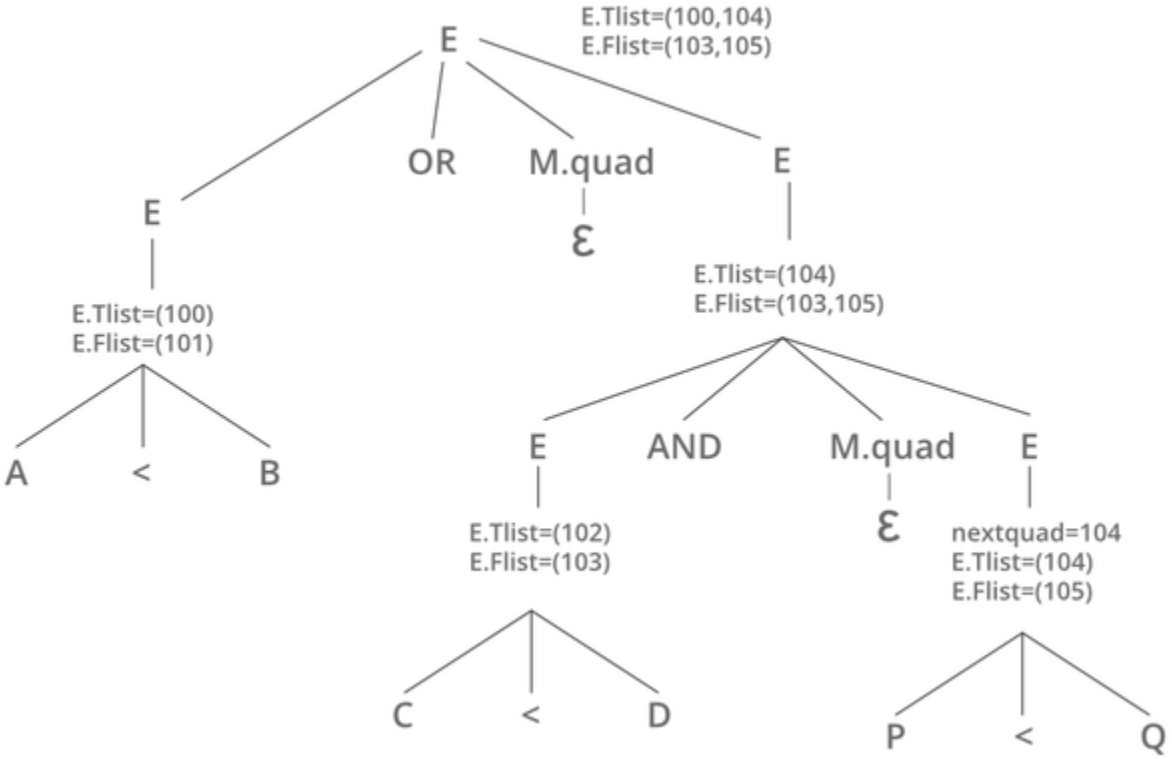
**Step 2:** We have to find the TAC(Three address code) for the given expression using backpatching:

**A < B OR C < D AND P < Q**




*Three address codes for the given example*

**Step 3:** Now we will make the parse tree for the expression:




*Parse tree for the example*

## The flow of Control Statements:

Control statements are those that alter the order in which statements are executed. If, If-else, Switch-Case, and while-do statements are examples. Boolean expressions are often used in computer languages to

- **Alter the flow of control:** Boolean expressions are conditional expressions that change the flow of control in a statement. The value of such a Boolean statement is implicit in the program's position. For example, if (A) B, the expression A must be true if statement B is reached.
- **Compute logical values:** During bottom-up parsing, it may generate code for Boolean statements via a translation mechanism. A non-terminal marker M in the grammar establishes a semantic action that takes the index of the following instruction to be formed at the appropriate moment.

## Applications of Backpatching:

- Backpatching is used to translate flow-of-control statements in one pass itself.
- Backpatching is used for producing quadruples for boolean expressions during bottom-up parsing.
- It is the activity of filling up unspecified information of labels during the code generation process.
- It helps to resolve forward branches that have been planted in the code.

# LECTURE- 9

# REPRESENTING TAC USING QUADRUPLE, TRIPLES, INDIRECT TRIPLES

## Implementation of Three Address Code

There are 3 representations of three address code namely

1. Quadruple
2. Triples
3. Indirect Triples

## 1. Quadruple

It is structure with consist of 4 fields namely op, arg1, arg2 and result. op denotes the operator and arg1 and arg2 denotes the two operands and result is used to store the result of the expression.

**Advantage**

- Easy to rearrange code for global optimization.
- One can quickly access value of temporary variables using symbol table.

**Disadvantage**

- Contain lot of temporaries.
- Temporary variable creation increases time and space complexity.

- **Example –** Consider expression a = b * – c + b * – c.
  The three address code is:

```
t1 = uminus c
t2 = b * t1
t3 = uminus c
t4 = b * t3
t5 = t2 + t4
a = t5
```

| # | Op | Arg1 | Arg2 | Result |
|---|---|---|---|---|
| (0) | uminus | c | | t1 |
| (1) | * | t1 | b | t2 |
| (2) | uminus | c | | t3 |
| (3) | * | t3 | b | t4 |
| (4) | + | t2 | t4 | t5 |
| (5) | = | t5 | | a |

Quadruple representation

## 2. Triples

This representation doesn't make use of extra temporary variable to represent a single operation instead when a reference to another triple's value is needed, a pointer to that triple is used. So, it consist of only three fields namely op, arg1 and arg2.

### Disadvantage

- Temporaries are implicit and difficult to rearrange code.
- It is difficult to optimize because optimization involves moving intermediate code. When a triple is moved, any other triple referring to it must be updated also. With help of pointer one can directly access symbol table entry.

## Example

Consider expression a = b * – c + b * – c

| # | Op | Arg1 | Arg2 |
|---|---|---|---|
| (0) | uminus | c | |
| (1) | * | (0) | b |
| (2) | uminus | c | |
| (3) | * | (2) | b |
| (4) | + | (1) | (3) |
| (5) | = | a | (4) |

Triples representation

## 3.Indirect Triples

This representation makes use of pointer to the listing of all references to computations

which is made separately and stored. Its similar in utility as compared to quadruple representation but requires less space than it. Temporaries are implicit and easier to rearrange code.

**Example –** Consider expression a = b * – c + b * – c

| # | Op | Arg1 | Arg2 |
|---|---|---|---|
| (14) | uminus | c | |
| (15) | * | (14) | b |
| (16) | uminus | c | |
| (17) | * | (16) | b |
| (18) | + | (15) | (17) |
| (19) | = | a | (18) |

List of pointers to table

| # | Statement |
|---|---|
| (0) | (14) |
| (1) | (15) |
| (2) | (16) |
| (3) | (17) |
| (4) | (18) |
| (5) | (19) |

**Indirect Triples representation**

# LECTURE-10
# BOOLEAN EXPRESSIONS

Boolean expressions have two primary purposes. They are used to compute logical values, but more often they are used as conditional expressions in statements that alter the flow of control, such as if-then-else, or while-do statements.

Boolean expressions are composed of the boolean operators ( and, or, and not ) applied to elements that are boolean variables or relational expressions. Relational expressions are of the form E1 relop E2, where E1 and E2 are arithmetic expressions.

Here we consider boolean expressions generated by the following grammar :
E->EorE | EandE |notE | ( E ) |id relop id | true | false

## Methods of Translating Boolean Expressions:

There are two principal methods of representing the value of a boolean expression. They are :

* To encode true and false numerically and to evaluate a boolean expression analogously to an arithmetic expression. Often, 1 is used to denote true and 0 to denote false.

* To implement boolean expressions by flow of control, that is, representing the value of a boolean expression by a position reached in a program. This method is particularly convenient in implementing the boolean expressions in flow-of-control statements, such as the if-then and while-do statements.

## Numerical Representation

Here, 1 denotes true and 0 denotes false. Expressions will be evaluated completely from left to right, in a manner similar to arithmetic expressions.
For example :

* The translation for a or b and not c is the three-address sequence
*
```
t1 : = not c
t2 : = b and t1
t3 : = a or t2
```

* A relational expression such as a < b is equivalent to the conditional statement
* if a < b then 1 else 0

which can be translated into the three-address code sequence (aga statement numbers at 100) :

```
100 : if a < b goto 103
101 : t : = 0
102 : goto 104
103 : t : = 1
104 :
```

## Translation scheme using a numerical representation for Booleans

```
E→E1 orE2        { E.place : = newtemp;
                   emit( E.place ': =' E1.place 'or' E2.place ) }
E→E1 andE2       { E.place : = newtemp;
                   emit( E.place ': =' E1.place 'and' E2.place ) }
E→notE1          { E.place : = newtemp;
                   emit( E.place ': =' 'not' E1.place ) }
E→ ( E1)         { E.place : = E1.place }

E→id1 relop id2  { E.place : = newtemp;
      emit( 'if' id1.place relop.op id2.place 'goto' nextstat + 3); emit( E.place ': =' '0' );
                   emit('goto' nextstat +2);
                   emit( E.place ': =' '1') }
E→true           { E.place : = newtemp;
                   emit( E.place ': =' '1') }
E→false          { E.place : = newtemp;
                   emit( E.place ': =' '0') }
```

## Short-Circuit Code:

We can also translate a boolean expression into three-address code without generating code for any of the boolean operators and without having the code necessarily evaluate the entire expression. This style of evaluation is sometimes called "short-circuit" or "jumping" code. It is possible to evaluate boolean expressions without generating code for the boolean operators and, or, and not if we represent the value of an expression by a position in the code sequence.

Translation of a < b or c < d and e < f

```
100 : if a < b goto 103
101 : t1 : = 0
102 : goto 104
```

103 : t1 : = 1

104 : if c < d goto 107
105 : t2 : = 0
106 : goto 108
107 : t2 : = 1

108 : if e < f goto 111 109 : t3 : = 0
110 : goto 112
111 : t3 : = 1
112 : t4 : = t2 and t3
113 : t5 : = t1 or t4

## SDT for Switch Case

### • Syntax for switch case statement

switch E
begin
case V1: S1
case V2: S2
…
case Vn-1: Sn-1
default: Sn
end

### • Syntax Directed Translation

code to Evaluate E into t
goto Ltest
L1: code for S1
goto Lnext
L2: code for S2
goto Lnext
...
Ln-1: code for Sn-1
goto Lnext
Ln: code for Sn
goto Lnext
Ltest: if t = V1 goto L1
if t = V2 goto L2
…
if t = Vn-1 goto Ln-1
goto Ln
Lnext:

# SDT for Procedure Call

• For a function fun with n arguments a1,a2,a3….an i.e., fun(a1, a2, a3,…an)

## • Syntax Directed Translation

codeGen_expr (E):
codeGen_expr_list (arguments);

E.place = newtemp (f.returnType);
E.code = code to evaluate the arguments
Param a1
Param a2
…
Param an
Call fun, n …
retrieve E.place;
Where param defines the arguments to function.